SE446: Big Data Engineering

Week 8A: Spark on the Cluster — Deployment & Performance Tuning

Prof. Anis Koubaa

SE 446
Alfaisal University
https://github.com/aniskoubaa/big_data_course

Spring 2026

Instructor Notes

Welcome everyone to Week 8A. Today's theme is all about moving from the playground to production. Up until now, you have been running Spark interactively in a shell or in Jupyter notebooks, and that is perfectly fine for exploring data. But starting today, we are going to learn how to submit real jobs to a cluster and, more importantly, how to make those jobs fast. Think of it this way: last week you learned to drive, and today we are going to learn how to drive on the highway without crashing. By the end of this session, you will have the skills you need to write production-quality Spark code for your M3 milestone. So let us get started.

Today's Agenda

Instructor Notes

Let me give you a quick roadmap of where we are headed today. We have four big topics to cover. First, we will do a brief recap and see where we are on our Big Data journey. Second, we will learn how to deploy jobs to the cluster using `spark-submit`, including deploy modes and resource configuration. Third, we will dive into the Spark Web UI so you can diagnose what is happening inside your jobs. And finally, the bulk of the lecture is about performance tuning — caching, partitioning, and broadcast joins. These are the three levers that will make the difference between a job that takes ten minutes and one that takes two.

Outline

Learning Objectives

By the end of this session you will be able to:

1. Submit Spark jobs to a YARN cluster using `spark-submit`
2. Distinguish `client` vs `cluster` deploy modes
3. Read the Spark Web UI: Jobs, Stages, Tasks, Storage, Executors
4. Apply caching with `.cache()` and `.persist()`
5. Tune partition count for optimal parallelism
6. Use broadcast joins for small lookup tables

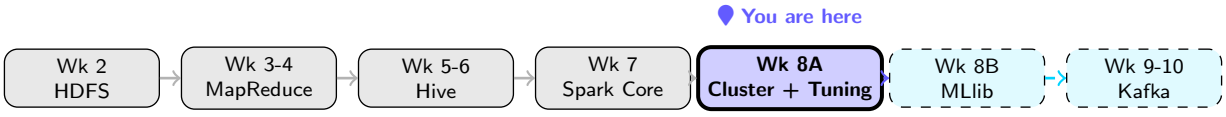
Dataset

Chicago Crimes (HDFS) — same dataset from Week 7

Instructor Notes

Let us walk through these six objectives because every one of them maps directly to what you need for your M3 milestone. Objective one is about `spark-submit` — that is how you actually run code on the cluster instead of typing it into a notebook. Objective two, deploy modes, determines where your driver program lives, and getting this wrong is a very common source of confusion. Objective three, the Web UI, is your diagnostic dashboard — you cannot tune what you cannot measure. Objectives four through six are the three performance levers: caching, partitions, and broadcast joins. For M3, you will need all six of these skills, so pay close attention and ask questions as we go. We are using the same Chicago Crimes dataset from last week, so the data should already feel familiar.

The Big Data Journey — Where Are We?



What you already know

- RDDs, DataFrames, Spark SQL
- PySpark in **interactive shell**
- Ran jobs locally or on the shell

What changes today

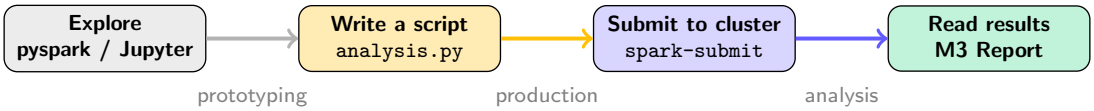
- Submit **real jobs** to the cluster
- Read the **Spark Web UI**
- Make jobs **fast** (tuning)
- **M3 milestone** uses these skills

Instructor Notes

Take a look at the timeline here. You can see we have come a long way since Week 2. You started with HDFS, learned how data is stored across a cluster. Then you wrote MapReduce jobs the hard way, which gave you an appreciation for distributed computing. Hive taught you SQL on top of Hadoop. Last week with Spark Core, you learned RDDs and DataFrames, which made everything much more pleasant. Now, right here at Week 8A, we are adding the final piece: running Spark on the actual cluster with real resource management and performance tuning. After today, you will be ready for MLlib in 8B, where we build machine learning models. So think of today as the bridge between knowing Spark and being able to use Spark effectively in the real world. Everything on the left side of this diagram is knowledge you already have, and we are building on top of it.

Outline

From Exploration to Submission — Your M3 Workflow



PySpark Shell / Jupyter

- Type code, see results instantly
- Great for **exploring** the dataset
- Driver runs on **your machine**
- Cannot run unattended

`spark-submit` on YARN

- Runs a `.py` script as a batch job
- Works on the **cluster's resources**
- Reproducible — versioned in GitHub
- Required for **M3 submission**

The Rule

Prototype in Jupyter $\longrightarrow$ polish to `.py` $\longrightarrow$ `spark-submit` $\longrightarrow$ submit results for M3.

Instructor Notes

This slide shows you the exact workflow you need to follow for M3, so pay attention. Step one, you explore in Jupyter or the PySpark shell — try things out, look at the data, experiment. Step two, once your analysis works, you copy the code into a clean Python file. Step three, you submit that file to the cluster using `spark-submit`. Step four, you collect the results for your report. This is exactly how data engineers work in industry: nobody runs production jobs from a notebook. The notebook is your scratchpad, the Python script is your deliverable. For M3, I will be checking that you used `spark-submit`, not just a notebook. So get comfortable with this flow during today's lab.

spark-submit Syntax

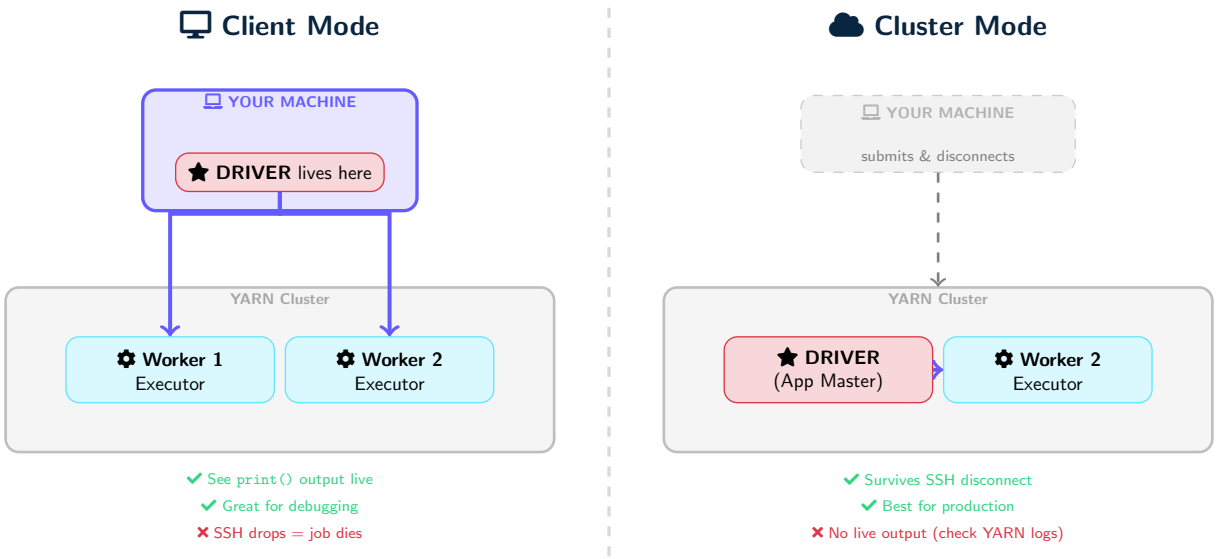
```
spark-submit \
    --master yarn \
    --deploy-mode client \
    --num-executors 2 \
    --executor-memory 1g \
    --executor-cores 2 \
    --driver-memory 512m \
    --conf spark.sql.shuffle.partitions=4 \
    crime_analysis.py
```

Flag	Purpose
`--master yarn`	Run on YARN cluster
`--deploy-mode`	Where Driver runs (client/cluster)
`--num-executors`	Number of executor JVMs
`--executor-memory`	RAM per executor
`--executor-cores`	CPU cores per executor
`--conf`	Custom Spark configuration

Instructor Notes

Let us walk through each flag in this command because you will be typing this many times. The `--master yarn` flag tells Spark to use YARN as the resource manager instead of running locally. The `--deploy-mode` decides where the driver lives, which we will cover in detail on the next slide. The `--num-executors 2` means we want two executor processes — one on each worker node in our cluster. Now here is the important one: `--executor-memory 1g` sets one gigabyte of RAM per executor. Why one gigabyte and not two? Because each worker node only has two gigabytes total, and YARN itself needs some overhead memory. If you request two gigabytes per executor, YARN will reject your job. The `--executor-cores 2` gives each executor two CPU cores for parallel task execution. Finally, the `--conf` flag lets you set any Spark configuration property, and we will see later why `shuffle.partitions` is critical.

Deploy Mode: `client` vs `cluster`



Instructor Notes

This is one of the most important diagrams to understand. Look at the left side first: in client mode, the Driver — which is the brain of your Spark application — lives on your machine. It sends instructions to the executors on the cluster. The advantage is that you see `print()` output directly in your terminal, which is fantastic for debugging. The disadvantage is that if your SSH connection drops, the driver dies, and the entire job is killed. Now look at the right side: in cluster mode, your machine just submits the job and walks away. The Driver runs inside the YARN cluster itself as the Application Master. Even if you close your laptop, the job keeps running. Here is my simple test for choosing: ask yourself, "If my SSH drops, is that okay?" If yes, use cluster mode. If you need to see output live, use client mode. For development, always start with client mode. For your final M3 submission of long-running jobs, switch to cluster mode.

Deploy Modes — Think of It Like a Phone Call

Client Mode = Speakerphone

- You're on the call the **entire time**
- You hear everything live (`print()` output)
- If you **hang up** (SSH drops), the call ends
- Best for: **development & debugging**

Cluster Mode = Voicemail

- You leave a message and **hang up**
- The work happens **without you**
- Check results later via YARN logs
- Best for: **production & long jobs**

Common Pitfall

Students often use `cluster` mode during development and wonder why they see no output. **Use `client` mode first**, switch to `cluster` for final submission.

Quick Check: You're debugging and need to see `print()` output. Which mode?

Deploy Modes — Think of It Like a Phone Call

Client Mode = Speakerphone

- You're on the call the **entire time**
- You hear everything live (`print()` output)
- If you **hang up** (SSH drops), the call ends
- Best for: **development & debugging**

Cluster Mode = Voicemail

- You leave a message and **hang up**
- The work happens **without you**
- Check results later via YARN logs
- Best for: **production & long jobs**

Common Pitfall

Students often use `cluster` mode during development and wonder why they see no output. **Use `client` mode first**, switch to `cluster` for final submission.

Quick Check: You're debugging and need to see `print()` output. Which mode?
Client mode!

Instructor Notes

Let me reinforce this with an analogy that I think makes it really click. Client mode is like being on speakerphone: you are there the whole time, you hear everything the other person says in real time, but if you hang up, the conversation is over. Cluster mode is like leaving a voicemail: you record your message, hang up, and the recipient deals with it on their own time. You check your messages later to see the response. Now here is the interactive part — I want you to think about this for a second. You are debugging your code and you need to see `print()` output immediately. Which mode do you use? Exactly, client mode. Every semester I get students who submit in cluster mode during development and then send me an email saying "my job produces no output." Do not be that student. Start with client, switch to cluster when you are confident the code works.

Setting Up Spark Client on Your Computer

1. Install Spark & Set Environment

Download the **same Spark version** as your cluster from `spark.apache.org`, then:

```
export SPARK_HOME=/opt/spark          # where you extracted Spark
export PATH=$SPARK_HOME/bin:$PATH
export HADOOP_CONF_DIR=/etc/hadoop/conf
```

2. Point to Your Cluster

Copy these three files from the cluster into `$HADOOP_CONF_DIR`: `core-site.xml`, `hdfs-site.xml`, `yarn-site.xml`. Your machine must be able to reach the cluster nodes over the network.

3. Submit in Client Mode

```
spark-submit --master yarn --deploy-mode client \
    --num-executors 2 --executor-memory 1g  my_analysis.py
```

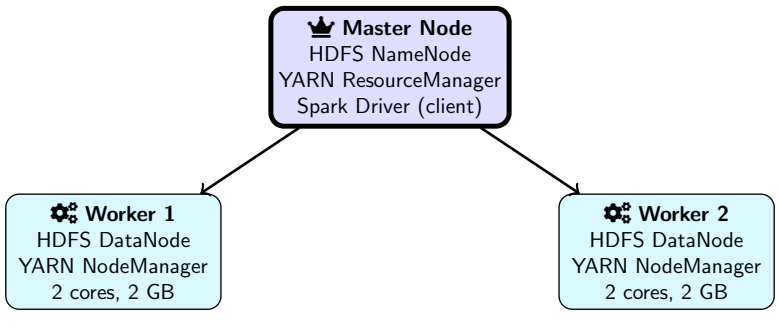
Your laptop is the **Driver** → `print()` output appears in your terminal.

⚠ Your machine must **stay connected** for the entire job. SSH drops = job dies. Use `cluster` mode for long unattended runs.

Instructor Notes

OK so now you have seen what client mode and cluster mode mean conceptually. Let me walk you through how to actually set this up on your own machine so you can work in client mode from your laptop. Look at the slide — three steps. Step one: install Spark locally. Go to `spark.apache.org` and download the exact same version that your cluster is running. This matters. If your cluster is on Spark 3.5 and you install 3.4 locally, you will get strange serialization errors when the Driver tries to talk to the executors. Extract it somewhere like `/opt/spark`, then set three environment variables: `SPARK_HOME` points to where you extracted it, add the `bin` folder to your `PATH` so you can type `spark-submit` from anywhere, and `HADOOP_CONF_DIR` tells Spark where to find the cluster configuration. Step two: you need the cluster's configuration files. There are three XML files: `core-site.xml` tells Spark where HDFS lives, `hdfs-site.xml` has the HDFS settings, and `yarn-site.xml` tells Spark how to reach YARN. Copy them from one of your cluster nodes and drop them into the `HADOOP_CONF_DIR` directory. Also make sure your laptop can actually talk to the cluster nodes — firewalls, VPNs, security groups, these are the things that trip people up. Step three: run `spark-submit` with `--deploy-mode client`. What happens? Your laptop becomes the Driver process. It contacts YARN, YARN launches executors on the cluster's worker nodes, and those executors talk back to your laptop. Because the Driver is on your machine, every `print()` shows up right in your terminal — great for development. But remember the warning at the bottom: your machine has to stay connected the entire time. If your WiFi drops or your laptop sleeps, the Driver is gone and YARN kills the executors. For anything long-running, switch to cluster mode so the job survives without you.

Our Cluster Configuration



Spark Web UI: `http://master:4040` — YARN UI: `http://master:8088`

Cluster Resources

Total: 4 cores, 4 GB RAM
Request $\leq$ 1g per executor

Analogy

Master = **project manager** (coordinates)
Workers = **team members** (do the work)

Instructor Notes

This is the actual cluster you will be using in the lab and for M3, so let me describe it concretely. We have one master node and two worker nodes. The master runs three key services: the HDFS NameNode that tracks where data blocks are stored, the YARN ResourceManager that allocates resources to applications, and when you use client mode, the Spark Driver also runs there. Each worker has two CPU cores and two gigabytes of RAM. That is it — four cores and four gigabytes total for computation. This is a teaching cluster, not a production cluster, so resources are tight. That is actually a good thing because it forces you to think about efficiency. In industry, you might have hundreds of nodes, but the tuning principles are exactly the same. Notice the two URLs at the bottom — you will use port 4040 for the Spark Web UI while your app is running, and port 8088 for the YARN UI to check completed applications.

Understanding the Cluster — Resource Math

Master = Coordinator

- **NameNode**: knows where data blocks are
- **ResourceManager**: allocates containers
- **Driver** (client): plans execution

Workers = Muscle

- **DataNode**: stores data blocks
- **NodeManager**: manages containers
- **Executors**: run your tasks

Resource Math

Total cluster: 4 cores, 4 GB. With `--num-executors 2 --executor-cores 2 --executor-memory 1g`, you use **all 4 cores** and **2 GB** (leaving headroom for OS and YARN overhead).

Instructor Notes

Let us do the math so you understand why we request the resources we do. Each worker has 2 GB of RAM, but not all of that is available to your executor. The operating system needs some memory, the YARN NodeManager daemon needs some, and YARN also adds an overhead buffer on top of what you request, typically around 384 megabytes. So if you request 1 gigabyte per executor, the actual container size is about 1.4 gigabytes, which fits within the 2 GB on each worker. If you try to request 2 gigabytes, the container would need roughly 2.4 gigabytes, and YARN will simply reject your application with a "container memory exceeds" error. For cores, we request 2 cores per executor times 2 executors, which gives us all 4 cores in the cluster. This means every core is busy when your job runs, which is exactly what we want. The key takeaway here is that you cannot just ask for all the RAM on a machine — you need to leave room for the system.

Example Script: Crime Counts

```
# crime_counts.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count

spark = SparkSession.builder \
    .appName("CrimeCounts") \
    .getOrCreate()

df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

result = df.groupBy("Primary Type") \
    .agg(count("*").alias("cnt")) \
    .orderBy(col("cnt").desc())

result.show(10)
spark.stop()
```

Submit to YARN

```
spark-submit --master yarn --deploy-mode client crime_counts.py
```

Instructor Notes

Let me walk you through this script line by line because it is the template you will use for M3. At the top, we import `SparkSession` and the functions we need. Then we use the builder pattern to create our Spark session — notice `.appName()` gives it a human-readable name that shows up in the YARN UI, and `.getOrCreate()` either creates a new session or reuses an existing one. Next, we read the CSV from HDFS — not from a local path, but from the distributed file system. The `groupBy("Primary Type")` groups all crime records by their type, and `.agg(count("*").alias("cnt"))` counts how many records are in each group. The `.orderBy(col("cnt").desc())` sorts them with the most common crime type first. Then `.show(10)` triggers the actual computation and prints the top ten results. Finally, and this is critical, `spark.stop()` releases all cluster resources. If you forget this line, your application keeps hogging memory and cores even after your analysis is done.

Think-Pair-Share: spark-submit

You have a script that takes 30 minutes to run.

Your laptop might lose WiFi during the run.

Which deploy mode should you use?

What flag do you add to `spark-submit`?

Discuss with your neighbor for 30 seconds.

Think-Pair-Share: spark-submit

You have a script that takes 30 minutes to run.

Your laptop might lose WiFi during the run.

Which deploy mode should you use?

What flag do you add to `spark-submit`?

Discuss with your neighbor for 30 seconds.

Answer

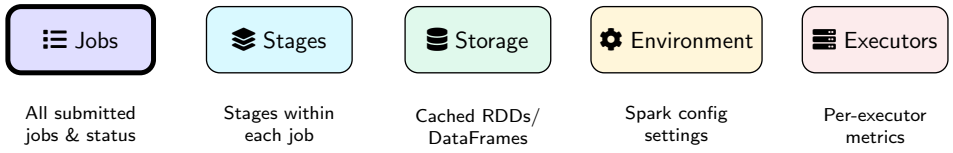
`--deploy-mode cluster` — the Driver runs inside YARN, so even if your WiFi drops, the job continues. Retrieve output with `yarn logs -applicationId <id>`.

Instructor Notes

Alright, take thirty seconds and discuss this with your neighbor. The scenario is simple: a job that runs for thirty minutes, and your WiFi is unreliable. Which deploy mode do you pick? Go ahead, talk it out. Okay, let us reveal the answer. You need `--deploy-mode cluster`. Why? Because in cluster mode, the driver runs inside the YARN cluster, not on your laptop. So even if your laptop loses WiFi, closes, or runs out of battery, the job keeps running happily on the cluster. When you reconnect later, you retrieve the output using `yarn logs -applicationId` followed by the application ID. Remember the phone call analogy — this is the voicemail scenario. You leave the message and walk away. The key flag to add is just `--deploy-mode cluster` instead of `client`. Simple change, but it can save you from losing thirty minutes of computation.

Outline

Spark Web UI — Overview



While App Runs

`http://master:4040`

After App Finishes

YARN proxy:
`http://master:8088/proxy/<app-id>/`

Key Principle

Each **action** creates a **job**. Each job has one or more **stages** (separated by shuffles). Each stage has multiple **tasks** (one per partition).

Instructor Notes

The Spark Web UI is your best friend when it comes to understanding what your job is actually doing. Let me describe each tab. The Jobs tab shows every job that has been submitted, its status — running, succeeded, or failed — and a timeline. The Stages tab is where you dig deeper: it shows you how each job is broken into stages, the duration of each stage, and how much data was shuffled. The Storage tab tells you what is currently cached in memory, which will be very useful when we talk about caching later. The Environment tab shows all your Spark configuration settings, so if something is misconfigured, you can check it there. The Executors tab gives you per-executor metrics like memory usage, CPU time, and garbage collection time. Now, an important distinction: while your application is running, you access the UI at port 4040 on the master. Once the application finishes, port 4040 disappears, and you need to go through the YARN proxy at port 8088. Keep both URLs bookmarked.

Predict the Execution Plan

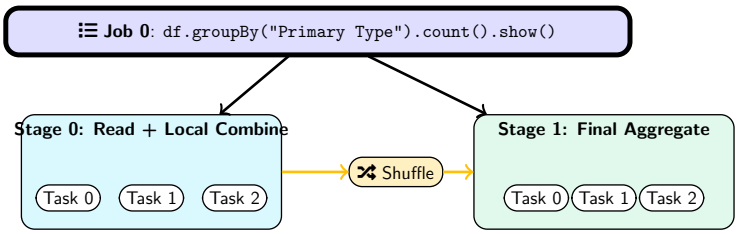
```
df.groupBy("Primary Type").count().show()
```

Before I show you the answer: How many **jobs**? How many **stages**? Why?
Hint: Which operation forces a shuffle?

Predict the Execution Plan

```
df.groupBy("Primary Type").count().show()
```

Before I show you the answer: How many **jobs**? How many **stages**? Why?
Hint: Which operation forces a shuffle?



Answer

1 job (one action: `.show()`), **2 stages** (separated by the `groupBy` shuffle).

Instructor Notes

Before I reveal the answer, I want you to think about this. We have `df.groupBy("Primary Type").count().show()`. How many jobs does this create? Remember the rule: one action equals one job. The only action here is `.show()`, so we get exactly one job. Now, how many stages? A new stage is created at every shuffle boundary. The `groupBy` operation requires a shuffle because Spark needs to bring all rows with the same Primary Type together onto the same partition. So we get two stages. Stage 0 reads the data from HDFS and does a local partial count on each partition — this is like each TA counting their own pile before combining. Then the shuffle moves data across the network, and Stage 1 does the final aggregation. Notice each stage has multiple tasks, one per partition. This DAG view is exactly what you will see in the Spark Web UI, so when you see two stages in the Jobs tab, you now know why.

Understanding Jobs, Stages, and Tasks

Jobs — Triggered by Actions

- Every **action** (`.show()`, `.count()`, `.collect()`, `.write`) creates a new **Job**
- Transformations (`.filter()`, `.groupBy()`) are lazy — they do **not** trigger Jobs

Stages — Separated by Shuffles

- A Job is split into **Stages** at every **shuffle boundary** (`groupBy`, `join`)
- Fewer stages = fewer shuffles = **better performance**

Tasks — One Per Partition

- Each stage launches **one task per partition** (4 partitions = 4 tasks)
- Tasks run in **parallel** across executor cores
- A slow task (straggler) slows the **entire stage** — check the Tasks tab!

Instructor Notes

Let me make this hierarchy crystal clear because it is the foundation for everything else. Think of it as a three-level pyramid. At the top, you have Jobs. The rule is simple: one action equals one job. Every time you call `.show()`, `.count()`, `.collect()`, or `.write`, Spark creates a new job. Transformations like `.filter()` and `.groupBy()` are lazy — they just build up a plan but do not execute anything. One level down, each job is divided into Stages. The dividing lines are shuffles: any operation that requires moving data across the network, like `groupBy` or `join`, creates a stage boundary. Fewer stages means fewer shuffles, which means better performance. At the bottom level, each stage contains Tasks. The rule here is one task per partition. If you have four partitions, you get four tasks. These tasks run in parallel across your executor cores. A critical thing to watch for is straggler tasks — if one task takes ten times longer than the others, it means you have a data skew problem, and the entire stage has to wait for that one slow task.

Spotting Performance Problems in the Web UI

Symptom	Where to Look	Cause & Fix
One task 10× slower	Tasks tab — Duration	**Data skew**: one partition too large. Fix: `repartition()` or salting.
High GC Time	Executors tab	**Memory pressure**: increase `executor-memory` or reduce partition size.
Large shuffle spill	Stages tab — Spill column	**Not enough RAM**: data spills to disk. Fix: more memory or fewer partitions.
Many empty tasks	Stages tab — Task count	**Over-partitioned**: too many partitions. Fix: `coalesce()`.

Lab Task

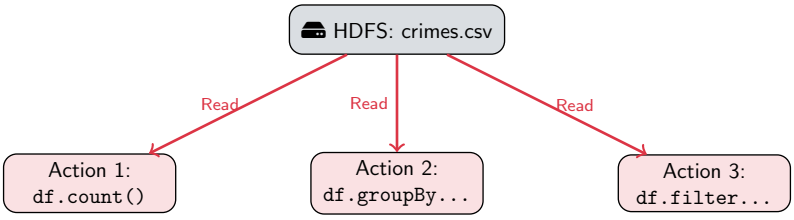
In today's lab, you'll observe each of these in the real Spark Web UI while running experiments on the cluster.

Instructor Notes

Let me go through each row in this table with a concrete example so you know what to look for. First symptom: one task taking ten times longer than others. Go to the Tasks tab and sort by duration. If you see most tasks finishing in two seconds but one taking twenty seconds, you have data skew — one partition has way more data than the others. The fix is `repartition()` to redistribute the data more evenly. Second symptom: high GC time. Go to the Executors tab and look at the GC Time column. If garbage collection is eating more than ten percent of your executor's time, you are running out of memory. Either increase `executor-memory` or make your partitions smaller so less data is processed at once. Third: large shuffle spill. In the Stages tab, look at the Spill columns. Spill means data that could not fit in memory during a shuffle, so Spark wrote it to disk temporarily. This is very slow. The fix is more memory or fewer partitions. Fourth: many empty tasks. If you see a hundred tasks but most of them process zero rows, you have too many partitions. Use `coalesce()` to reduce the count. You will see all of these in today's lab.

Outline

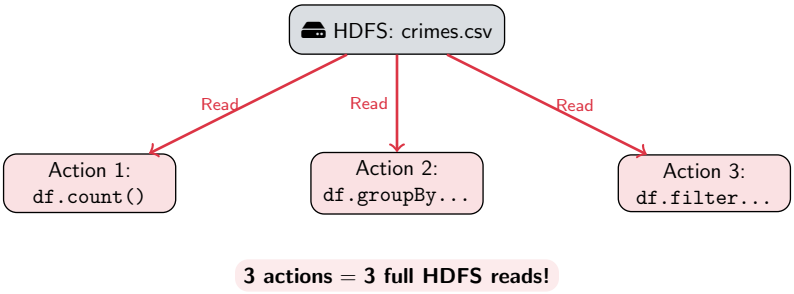
The Problem: Repeated HDFS Reads



3 actions = 3 full HDFS reads!

Quiz: You run 5 actions without caching. How many HDFS reads?

The Problem: Repeated HDFS Reads



3 actions = 3 full HDFS reads!

Quiz: You run 5 actions without caching. How many HDFS reads? **5 times!**

Why?

Spark is **lazy** — every action recomputes from scratch by replaying the DAG lineage.

Instructor Notes

Before I show you the solution, I want you to deeply understand the problem, because the solution will not make sense otherwise. Look at this diagram. You have one CSV file on HDFS, and your script runs three actions on it: a `count()`, a `groupBy`, and a `filter`. How many times does Spark read that file from HDFS? Three times. The full file, from disk, across the network, three times. Why? Because Spark is lazy. It does not remember the result of a previous computation. Every time you call an action, Spark replays the entire DAG from scratch — starting from the original data source. Now here is the quiz: if you run five actions without caching, how many HDFS reads happen? Exactly, five. On our small dataset, each read might take only a few seconds, but imagine a production dataset with billions of rows — each read could take minutes. That is minutes of wasted time re-reading data you already had in memory. This is the number one performance mistake students make.

Why Does Spark Re-Read? — The Library Analogy

✗ Without Caching

- Like walking to the shelf **every time** you need a fact
- Need the population of Riyadh? Walk to shelf.
- Need it again? Walk **again**.
- 5 questions = 5 trips

✓ With Caching

- Like **photocopying** the page
- First trip: walk + photocopy
- Every question after: read from **your desk**
- 5 questions = 1 trip + 4 instant reads

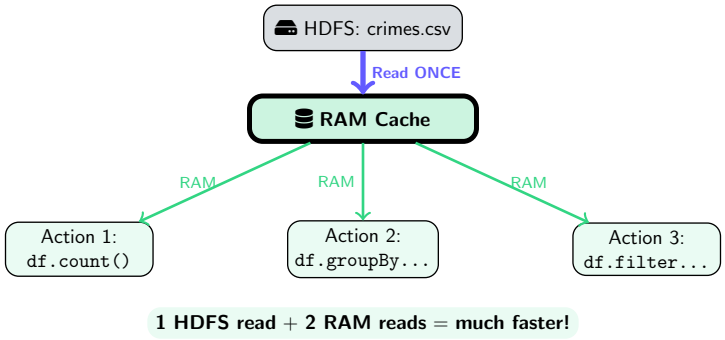
The Cost in Practice

- `df.count()` — reads 10,000 rows from HDFS
- `df.groupBy(...).show()` — reads the **same** 10,000 rows again
- With real datasets (millions of rows), each read can take **minutes**

Instructor Notes

Let me use an analogy to make Spark's lazy evaluation really click. Imagine you are in a library doing a research assignment. Without caching, every time you need a fact from a book, you walk to the shelf, find the book, look up the fact, walk back to your desk, and write it down. Then when you need another fact from the same book, you walk all the way back to the shelf again. Five questions, five trips. That is what Spark does without caching — it goes back to HDFS every single time. Now, with caching, it is like photocopying the page on your first trip. You walk to the shelf once, photocopy the relevant page, bring it back to your desk, and now every subsequent question is answered instantly from the photocopy on your desk. Five questions, but only one trip to the shelf plus four instant reads from your desk. The photocopy is the cache — it costs a little extra effort the first time, but it saves enormous amounts of time on every subsequent access. This connects directly back to what we just discussed about HDFS reads.

The Solution: `.cache()` and `.persist()`



How It Works

1. `df.cache()` **marks** for caching (lazy!)
2. First action: reads HDFS, stores in RAM
3. Next actions: from **RAM** — 10–100× faster

Lab 07 Benchmark

Without cache: 21s + 21s + 21s = **63s**
With cache: 21s + 0.8s + 3.3s = **25s**
Speedup: **2.5**×

Instructor Notes

Now look at how caching changes the picture. Instead of three arrows going from HDFS to each action, we have one arrow from HDFS to the RAM cache, and then three arrows from RAM to each action. The key phrase in this diagram is “Read ONCE.” HDFS is only accessed on the first action. After that, every subsequent action reads from RAM, which is ten to a hundred times faster than reading from disk across the network. Look at the benchmark numbers from our lab: without caching, three actions took 63 seconds total because each one re-read the data at 21 seconds per read. With caching, the first action still took 21 seconds because it has to do the initial read and populate the cache, but the second action dropped to 0.8 seconds and the third to 3.3 seconds. Total: 25 seconds. That is a 2.5 times speedup, and this is on a small dataset. On larger datasets, the speedup is even more dramatic because HDFS reads scale with data size while RAM reads stay nearly instant.

Caching in Code

```
# .cache() = persist in RAM only (default)
df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)
df.cache()           # marks for caching (lazy!)
df.count()           # 1st action: reads HDFS, stores in RAM
df.groupBy("Primary Type").count().show()  # from RAM!

# Unpersist when done (free memory)
df.unpersist()
```

Common Mistake

`df.cache()` does **NOT** cache immediately! Caching happens on the **first action**.

Memory Tip

If DataFrame > RAM, Spark evicts partitions. Use `.persist(MEMORY_AND_DISK)` instead.

Instructor Notes

Let me walk you through this code carefully. First, we read the CSV from HDFS as usual. Then we call `df.cache()`. Now here is the critical point I want to hammer home: `cache()` does NOT actually cache anything at that moment. It is lazy, just like transformations. All it does is mark the DataFrame with a flag that says "when you compute this for the first time, keep the result in memory." The actual caching happens when you trigger the first action, which in this case is `df.count()`. That first action reads from HDFS, computes the result, and stores the DataFrame in executor memory. From that point on, every subsequent action, like the `groupBy` and `show()`, reads from RAM instead of HDFS. Finally, when you are done with the DataFrame, call `df.unpersist()` to free the memory. This is especially important on our small cluster where memory is limited. The number one mistake students make is calling `cache()` and then wondering why their first action is not faster — it is not faster because the first action is when the caching actually happens.

Storage Levels — .persist()

```python
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)   # spill to disk
df.persist(StorageLevel.DISK_ONLY)         # disk only
```

Level	RAM	Disk	Repl.	Use Case
MEMORY_ONLY	✓	✗	No	Default (.cache()). Fastest.
MEMORY_AND_DISK	✓	✓	No	Large data that may not fit
DISK_ONLY	✗	✓	No	Very large data, save RAM
MEMORY_ONLY_2	✓	✗	2×	Critical data, fault tolerance

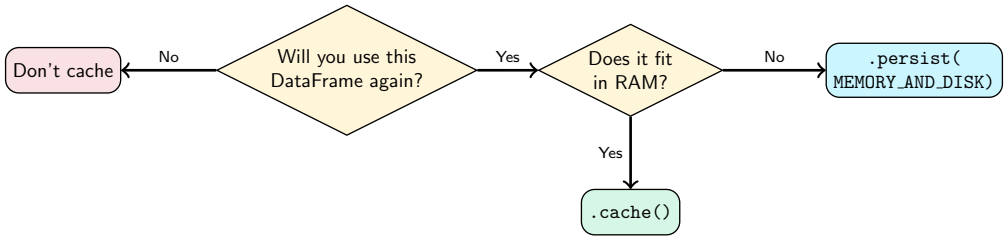
Rule of Thumb

Start with .cache() (MEMORY_ONLY). If you get OutOfMemory, switch to MEMORY_AND_DISK.

Instructor Notes

Now let us talk about the different storage levels, because .cache() is actually just a shortcut for .persist(StorageLevel.MEMORY_ONLY). In practice, most of you will only need two levels. The first is MEMORY_ONLY, which is the default when you call .cache(). It stores everything in RAM and is the fastest option. If a partition does not fit in memory, Spark simply does not cache that partition and recomputes it on the fly. The second level you might use is MEMORY_AND_DISK. This one stores as much as possible in RAM, and any partitions that do not fit get spilled to the executor's local disk. It is slower than pure RAM for the spilled partitions, but at least you do not have to recompute anything. DISK_ONLY is rarely used in practice because if you are reading from disk anyway, it is only marginally better than reading from HDFS. And MEMORY_ONLY_2 replicates the data across two executors for fault tolerance, which you typically only need in production environments. My recommendation: start with .cache() and only switch to MEMORY_AND_DISK if you get out-of-memory errors.

When to Cache — Decision Flowchart



Cache These

- DataFrames used by **multiple actions**
- **Training data** before `model.fit()`
- DataFrames after **expensive joins**

Don't Forget!

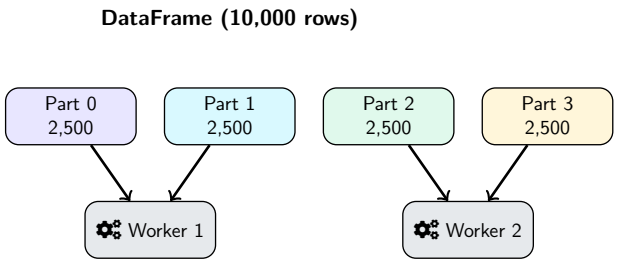
Call `df.unpersist()` when done. Cached data stays in memory until the app ends or you free it.

Instructor Notes

This decision flowchart should be your mental checklist every time you consider caching. Start at the first diamond: will you use this DataFrame again? If no, do not cache it. Caching a DataFrame you only use once is pure waste — it takes extra time to store the data in memory and you never get that investment back. If yes, move to the second question: does it fit in RAM? On our cluster, each executor has about one gigabyte, so if your DataFrame is smaller than that, use `.cache()`. If it is larger, use `.persist(MEMORY_AND_DISK)` so the overflow goes to disk instead of being evicted. Here are the most common cases where caching pays off: when you run multiple actions on the same DataFrame, like a count followed by a groupBy followed by a filter. When you are about to train a machine learning model, because `model.fit()` iterates over the training data many times internally. And when you have done an expensive join and want to reuse the result. Just remember to call `unpersist()` when you are done, especially on our small cluster where memory is precious.

Outline

What Are Partitions?



Analogy

4 TAs, 100 exams. Give each TA 25 = fast. Give 1 TA all 100 = slow. Partitions split data across cores the same way.

```
# Check partitions
df.rdd.getNumPartitions()
```

Key Rule

1 partition = 1 task. Each task runs on one core of one executor.

Instructor Notes

Let me explain partitions using an analogy you can all relate to. Imagine you have a hundred exams to grade and four TAs. If you give each TA twenty-five exams, they all work in parallel and finish quickly. But if you give all hundred exams to one TA and the other three sit idle, it takes four times as long. Partitions work the same way. A partition is simply a chunk of your DataFrame. When Spark processes data, it assigns one task per partition, and each task runs on one core. So if you have four partitions and four cores, all cores are busy simultaneously. The key rule to remember is: one partition equals one task. You can check how many partitions your DataFrame has by calling `df.rdd.getNumPartitions()`. In the diagram, you can see our ten thousand rows split into four partitions of 2,500 rows each, with two partitions assigned to each worker. This gives us perfect parallelism across our four cores.

The Goldilocks Problem — Finding the Right Partition Count

Too Few (1 partition)
1 core works
3 cores sit idle
SLOW

Just Right (8)
All 4 cores busy
2 rounds of work
FAST

Too Many (200)
Scheduling overhead
Mostly empty tasks
SLOW

The Formula

Sweet spot = 2–4× total executor cores. Our cluster: 4 cores → **8–16 partitions** is ideal.

How Partitions Are Decided

Default comes from the data source: **HDFS** → one partition per 128 MB block; **CSV** → depends on file splits. Override with `repartition(n)` or `coalesce(n)`.

Instructor Notes

This is the Goldilocks problem: you need just the right number of partitions, not too few and not too many. Let me walk through all three cases. Too few partitions, say just one: only one core processes data while the other three sit idle. You are using twenty-five percent of your cluster. That is like having four TAs but only giving exams to one of them. Too many partitions, say two hundred on our four-core cluster: each partition has almost no data, but Spark still has to schedule each one as a separate task. The overhead of scheduling, serializing, and coordinating two hundred tiny tasks overwhelms the actual computation. It is like cutting a pizza into two hundred slices — you spend more time cutting than eating. The sweet spot is two to four times your total core count. Why the multiplier? Because if one task finishes slightly faster than another, Spark can immediately assign the next partition to that free core, keeping utilization high. For our cluster with four cores, that means eight to sixteen partitions. The formula is simple: take your total executor cores and multiply by two to four. That is your target partition count.

`repartition()` vs `coalesce()`

`repartition(n)`	`coalesce(n)`
Full shuffle (expensive)	No shuffle (narrow dependency)
Can increase or decrease	Can only **decrease**
Even distribution guaranteed	Some partitions may be larger
Use for: heavy computation	Use for: writing output files

```
# Increase partitions (full shuffle)
df = df.repartition(8)

# Decrease partitions (no shuffle -- just merge)
df_out = df.coalesce(2)

# Partition by column (co-locate related rows)
df = df.repartition(4, "District")
```

Sweet Spot

Set partitions to **2–4× total executor cores**. Our cluster: 4 cores → 8 partitions is a good default.

Instructor Notes

Now you know how many partitions you want, but how do you actually change the number? You have two tools: `repartition()` and `coalesce()`. The key difference is shuffle versus no shuffle. `repartition()` does a full shuffle, meaning it redistributes all the data across the network to create perfectly even partitions. It can increase or decrease the partition count. Use this when you need even data distribution for heavy computation, like before a `groupBy` or a `join`. `coalesce()` does not shuffle at all — it simply merges adjacent partitions on the same executor. Because of this, it can only decrease the partition count, not increase it. The partitions may be slightly uneven, but it is much faster because no data moves across the network. Use `coalesce()` when you are about to write output files and want to control how many files are created. For example, `coalesce(2)` before a write gives you two output files instead of eight. A third option is `repartition(4, "District")`, which partitions by a specific column so all rows with the same district end up in the same partition — useful before joins on that column.

Shuffle Partitions — The #1 Setting to Change

After any **shuffle** (GROUP BY, JOIN, etc.), Spark creates `spark.sql.shuffle.partitions` output partitions — default is **200**.

```
# BAD for small clusters: 200 tasks on 4 cores!
spark.conf.set("spark.sql.shuffle.partitions", 8)
```

200 partitions on 4 cores

200 ÷ 4 = **50 rounds**
10,000 rows ÷ 200 = 50 rows each
Scheduling overhead ≫ actual work

8 partitions on 4 cores

8 ÷ 4 = **2 rounds**
10,000 rows ÷ 8 = 1,250 rows each
Each core stays busy — efficient!

Always Set This!

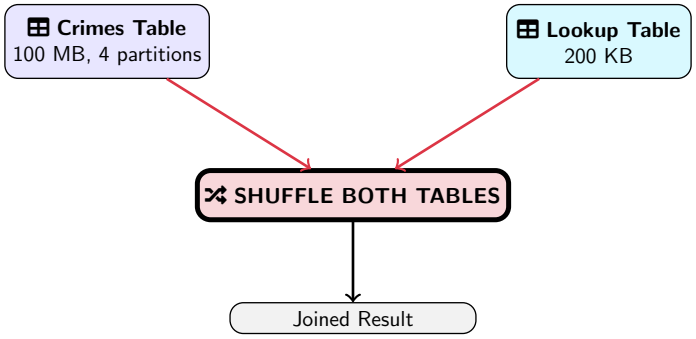
Add `--conf spark.sql.shuffle.partitions=8` to every `spark-submit` command on our cluster. This single setting can **halve** your job execution time.

Instructor Notes

This is the single most important configuration setting for our cluster, and it is the number one setting students forget. Here is the problem: every time Spark does a shuffle, like after a `groupBy` or a `join`, it creates a number of output partitions controlled by `spark.sql.shuffle.partitions`. The default is 200. That default makes sense for a large production cluster with hundreds of cores, but on our four-core cluster, it is a disaster. Let me do the math. With 200 partitions on 4 cores, Spark needs 50 rounds to process all partitions. Each partition only has 50 rows, so the actual work per task is tiny, but the overhead of scheduling, serializing, and coordinating each task is significant. Now compare: with 8 partitions on 4 cores, you need only 2 rounds. Each partition has 1,250 rows, which is enough data for meaningful work. I have seen students cut their job time in half just by adding this one setting. So here is what I want you to remember: always set `shuffle.partitions` to 8 on our cluster. Add it to every `spark-submit` command or set it in your `SparkSession.builder.config()`.

Outline

The Shuffle Join Problem



Shuffles 100 MB + 200 KB across network = WASTEFUL

Problem

Default join shuffles **both** tables. Shuffling 100 MB of crimes data just to join with a 200 KB lookup table is extremely wasteful.

Instructor Notes

Now let us set up the problem that broadcast joins solve. Imagine you have your Crimes table, which is 100 megabytes spread across four partitions, and you want to join it with a small lookup table that maps district IDs to district names — just 200 kilobytes. By default, Spark does a shuffle join, which means it shuffles both tables across the network so that matching keys end up on the same partition. Think about how wasteful this is: you are moving 100 megabytes of crime data across the network just to match it with a tiny 200-kilobyte table. That is like rearranging an entire library just to add a sticky note to a few books. The network transfer alone can take significant time, and on our small cluster with limited bandwidth, this is even worse. The total data shuffled is 100.2 megabytes, but the lookup table is only 0.2 percent of that. There has to be a better way, and there is.

Broadcast Join — The Formula Sheet Analogy

✗ Shuffle Join

Like making **every student** line up at **one shared** formula sheet taped to the wall.

Everyone moves, everyone waits, chaos ensues. (= network shuffle)

✓ Broadcast Join

Like **photocopying** the 1-page formula sheet and giving a copy to **every student**.

Students stay seated, work in parallel. (= local join)

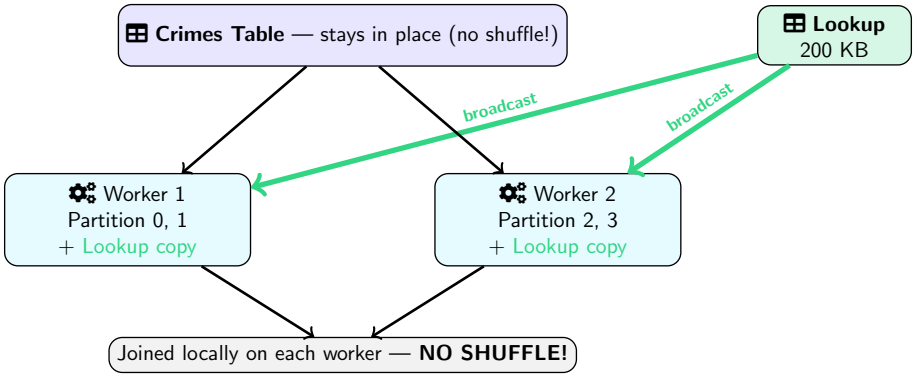
How It Works

- The small Lookup table (200 KB) is **copied entirely** to every executor
- The large Crimes table **stays in place** — partitions don't move
- Each executor joins locally — **no shuffle!**
- Network savings: **99.6% less data** transferred

Instructor Notes

Here is the analogy that makes this click. Imagine an exam room where students need a formula sheet to answer questions. The shuffle join approach is like taping one formula sheet to the wall and making every student line up, walk to the wall, look up their formula, walk back, and answer their question. Everyone is moving, everyone is waiting, it is chaos. That is what a shuffle join does — it moves all the data across the network. The broadcast join approach is like photocopying that one-page formula sheet and handing a copy to every student at their desk. The students never move. They look up formulas from their own copy and do their work in parallel. The formula sheet is small, so copying it twenty or thirty times is cheap. That is exactly what a broadcast join does: it copies the small lookup table to every executor. The large Crimes table stays in place, no partitions move across the network. Each executor joins its local partitions with its local copy of the lookup table. The network savings are enormous — 99.6 percent less data transferred in our example.

Broadcast Join — The Solution (Diagram)



Result

Small table (<10 MB) is copied to every executor. Join happens **locally** — zero shuffle of the large table!

Instructor Notes

Let me walk through this diagram carefully because it is the key to understanding broadcast joins visually. At the top, you see the Crimes table on the left — notice the label says "stays in place, no shuffle." That is the whole point. On the right, you have the small Lookup table at 200 kilobytes. The green arrows labeled "broadcast" show the lookup table being copied to both workers. So Worker 1 has partitions zero and one of the Crimes table plus a complete copy of the Lookup table. Worker 2 has partitions two and three plus its own copy of the Lookup table. Now each worker can perform the join locally by scanning its crime partitions and looking up district names from its local copy. No data from the Crimes table ever crosses the network. The only data that moves is the 200-kilobyte lookup table, which is copied twice — once to each worker. Compare that to the shuffle join where 100 megabytes would have been shuffled. The result at the bottom is produced by combining the local join results from both workers. This is how you get a 2.9 times speedup on joins with small tables.

Broadcast Join in Code

```python
from pyspark.sql.functions import broadcast

# Small lookup table
district_df = spark.createDataFrame([
    (1, "Central"), (2, "Wentworth"),
    (8, "Chicago Lawn"), (11, "Harrison")
], ["DistrictID", "DistrictName"])

# WITHOUT broadcast (shuffles both tables)
result = df.join(district_df,
    df["District"] == district_df["DistrictID"])

# WITH broadcast (sends lookup to all executors)
result = df.join(broadcast(district_df),
    df["District"] == district_df["DistrictID"])
```

Auto-Broadcast

Spark auto-broadcasts tables < 10 MB

Verify with explain()

Look for `BroadcastHashJoin` in `result.explain()`

Instructor Notes

Let me walk through this code. First, we import the `broadcast` function from `pyspark.sql.functions`. Then we create a small lookup DataFrame with district IDs and names — just four rows. The first join, without broadcast, is the default shuffle join: Spark will shuffle both the large Crimes DataFrame and the small district DataFrame across the network. The second join wraps `district_df` in the `broadcast()` function. This tells Spark to copy the small table to every executor instead of shuffling. The syntax change is minimal — you just wrap the small DataFrame in `broadcast()` — but the performance impact is huge. Now, an important detail: Spark actually auto-broadcasts tables smaller than 10 megabytes by default, controlled by the `spark.sql.autoBroadcastJoinThreshold` setting. But I recommend always being explicit with the `broadcast()` call so your intent is clear. To verify it worked, call `result.explain()` and look for `BroadcastHashJoin` in the output. If you see `SortMergeJoin` instead, the broadcast did not happen.

Join Strategy Summary

Scenario	Strategy	Cost
Both tables large	Shuffle (Sort-Merge) Join	High (full shuffle)
One table small (<10 MB)	**Broadcast Join**	Low (no shuffle)
Same partition key	Co-partitioned join	Medium (pre-repartition)
Frequent joins on same key	Bucket join	Low (pre-bucketed)

Key Insight

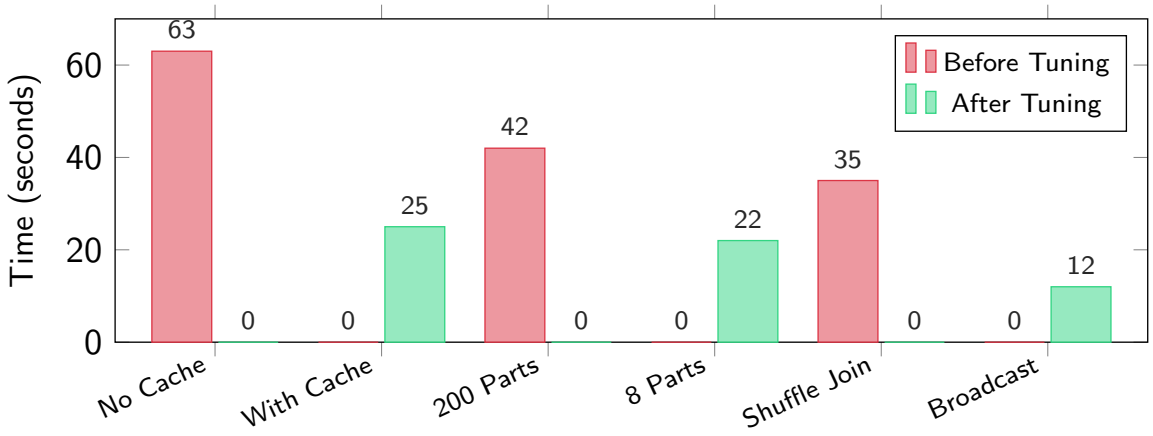
In Big Data courses, most "lookup" joins (district names, payment types, status codes) involve a **small dimension table**. Always use `broadcast()` for these!

Instructor Notes

Let me quickly recap when to use which join strategy so you have a clear decision framework. If both tables are large — say, joining two multi-gigabyte tables — you have no choice but to use a shuffle join, also called a sort-merge join. It is expensive because both tables get shuffled, but there is no way around it. If one table is small, under 10 megabytes, always use a broadcast join. This is the most common scenario you will encounter, especially with lookup tables like district names, crime categories, or status codes. The third and fourth options — co-partitioned and bucket joins — are more advanced. Co-partitioned means both tables are already partitioned on the join key, so Spark can join them partition by partition without a full shuffle. Bucket joins go further by pre-sorting data into buckets on disk. You will not need these for M3, but they are good to know about. For your coursework, the key takeaway is: whenever you join with a small dimension table, wrap it in `broadcast()`.

Outline

Performance Benchmarks — Real Numbers



Cache: 2.5× faster — **Partitions**: 1.9× faster — **Broadcast**: 2.9× faster

Combined: all three levers together can yield **5–10**× *speedup on real workloads.*

Instructor Notes

Let me walk you through this bar chart because these are real numbers from our cluster. The red bars show performance before tuning and the green bars show after tuning. First pair: without caching, three actions took 63 seconds. With caching, the same three actions took 25 seconds. That is a 2.5 times speedup from one line of code. Second pair: with the default 200 shuffle partitions, a groupBy query took 42 seconds. After setting shuffle partitions to 8, the same query took 22 seconds. That is a 1.9 times speedup from one configuration change. Third pair: a shuffle join took 35 seconds. A broadcast join on the same data took 12 seconds. That is a 2.9 times speedup. Now here is what makes this really powerful: these levers are not mutually exclusive. You can apply all three at the same time. In practice, combining caching, proper partitioning, and broadcast joins can yield a five to ten times overall speedup on real workloads. These are not theoretical numbers — you will reproduce them in today's lab.

Key Takeaways

1. **spark-submit**: deploy `.py` scripts to YARN (`client` for dev, `cluster` for production)
2. **Web UI**: Jobs → Stages → Tasks. Watch shuffle bytes, straggler tasks, GC time
3. **Caching**: `.cache()` keeps data in RAM; avoids repeated HDFS reads
4. **Partitions**: set to 2–4× total cores. Use `repartition()` to increase, `coalesce()` to decrease
5. **Broadcast joins**: send small tables to all executors; eliminates shuffle of the large table

The Three Levers of Spark Performance

Lever 1: Caching — Lever 2: Partitioning — Lever 3: Broadcast

Instructor Notes

Let me do a rapid review of everything we covered. Point one: `spark-submit` is how you deploy scripts to the cluster. Use client mode for development so you see output, cluster mode for production so the job survives disconnections. Point two: the Web UI at port 4040 is your diagnostic tool. The hierarchy is Jobs, Stages, Tasks — watch for shuffle bytes, straggler tasks, and garbage collection time. Point three: caching with `.cache()` keeps your DataFrame in RAM after the first action, eliminating repeated HDFS reads. Point four: set your partition count to two to four times your total cores. Use `repartition()` to increase, `coalesce()` to decrease, and always set `shuffle.partitions` to 8 on our cluster. Point five: broadcast joins copy small tables to every executor, eliminating the need to shuffle the large table. Remember the framework: three levers of Spark performance — caching, partitioning, and broadcast. Every performance problem you encounter can be addressed by one of these three levers.

Spark Tuning Cheat Sheet — Screenshot This!

spark-submit Template:

```
spark-submit --master yarn --deploy-mode client \
    --num-executors 2 --executor-memory 1g --executor-cores 2 \
    --conf spark.sql.shuffle.partitions=8 my_script.py
```

Caching	Partitions	Broadcast
Multiple actions on same DF?	Set to 2–4× total cores	Small table < 10 MB?
→ `df.cache()`	Our cluster: **8 partitions**	→ `broadcast(small_df)`
OOM? `.persist(M&D)`	`repartition(n)` / `coalesce(n)`	Verify: `result.explain()`
Done? `df.unpersist()`	Always set shuffle.partitions!	Never broadcast large tables

Web UI Quick Reference

While running: `http://master:4040` — After finish: `http://master:8088/proxy/<app-id>/`

Instructor Notes

I strongly recommend you screenshot this slide right now because it is your one-page reference for everything we covered today. At the top, you have the exact `spark-submit` template you will use for M3. Just replace `my_script.py` with your actual script name. The table below has three columns, one for each performance lever. The Caching column reminds you: if you use a DataFrame in multiple actions, call `.cache()`. If you get an out-of-memory error, switch to `.persist(MEMORY_AND_DISK)`. Always call `unpersist()` when done. The Partitions column reminds you: set to two to four times your total cores, which is 8 partitions for our cluster. Use `repartition()` to increase, `coalesce()` to decrease, and always set `shuffle.partitions`. The Broadcast column reminds you: for any table under 10 megabytes, wrap it in `broadcast()` and verify with `explain()`. At the bottom, you have the Web UI URLs. Keep this slide handy during the lab and while working on M3.

Top 5 Student Mistakes — Avoid These!

1. **Using cluster mode during development**
   You see no output → use `client` mode for debugging
2. **Expecting `.cache()` to work immediately**
   It's lazy! Caching happens on the first **action** after the call
3. **Leaving shuffle partitions at 200**
   200 tasks on 4 cores = 50 rounds of overhead. Set to 8!
4. **Forgetting `spark.stop()`**
   Your app hogs cluster resources. Always call it at the end.
5. **Broadcasting a large table**
   Causes `OutOfMemoryError` on every executor. Only broadcast < 10 MB.

The Fix Pattern

Client mode → cache if reused → set shuffle partitions to 8 → broadcast small tables → `spark.stop()` at the end.

Instructor Notes

Let me go through each of these mistakes with what actually happens when students make them, because I see all five every semester. Mistake one: using cluster mode during development. A student runs their script, waits five minutes, sees nothing on screen, and emails me saying "Spark is broken." It is not broken — the output is in the YARN logs because the driver is running on the cluster, not on your terminal. Fix: use client mode for development. Mistake two: expecting `.cache()` to speed up the very next action. A student calls `cache()` and then `count()`, and wonders why the count was not instant. The count is the action that triggers the caching — the speedup comes on the second action. Mistake three: leaving shuffle partitions at 200. The job takes forever and the student blames the cluster. In reality, Spark is spending more time scheduling 200 tiny tasks than doing actual work. Set it to 8. Mistake four: forgetting `spark.stop()`. The student's application keeps running on the cluster, consuming resources that other students need. This is especially bad on a shared cluster. Mistake five: broadcasting a 500-megabyte table. Spark tries to copy it to every executor, and they all run out of memory simultaneously. Only broadcast tables under 10 megabytes.

Putting It All Together — Optimized Script

```
# optimized_analysis.py -- applies all three tuning levers
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, broadcast

spark = SparkSession.builder \
    .appName("OptimizedCrimeAnalysis") \
    .config("spark.sql.shuffle.partitions", 8) \
    .getOrCreate()

df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)
df = df.repartition(8)      # Lever 2: match core count
df.cache()                  # Lever 1: reuse across actions

df.count()                  # triggers cache
top_crimes = df.groupBy("Primary Type") \
    .agg(count("*").alias("cnt")).orderBy(col("cnt").desc())
top_crimes.show(10)         # reads from RAM!

# Lever 3: broadcast small lookup table
districts = spark.createDataFrame([(1,"Central"),(2,"Wentworth")],
                                  ["id","name"])
joined = df.join(broadcast(districts), df["District"]==districts["id"])

df.unpersist()              # free memory
spark.stop()                # release cluster resources
```

Instructor Notes

This is the template for your M3 submission, so let me walk through it showing where each lever is applied. At the top, we build the SparkSession and set `shuffle.partitions` to 8 right in the config — that is lever two applied globally. Then we read the CSV from HDFS and immediately call `repartition(8)` to set our read partitions to match our core count — that is lever two again, this time for the input data. Next, `df.cache()` marks the DataFrame for caching — lever one. The `df.count()` call is intentional: it triggers the cache so that the subsequent `groupBy` and `show()` read from RAM instead of HDFS. Then we do the `groupBy` analysis, which benefits from both the cache and the 8 shuffle partitions. Further down, we create a small lookup table and join it using `broadcast()` — that is lever three. Finally, we clean up with `unpersist()` to free memory and `spark.stop()` to release cluster resources. This script applies all three levers and follows all the best practices. Use it as your starting point for M3.

What's Next

Week 8B: Spark MLlib — Machine Learning at Scale

- ML Pipeline API: `Transformer`, `Estimator`, `Pipeline`
- Feature engineering: `StringIndexer`, `VectorAssembler`
- Train a Random Forest to predict arrests in Chicago crimes
- Evaluate: AUC-ROC, accuracy, F1 score, confusion matrix

Today's Lab: Spark Performance Tuning

- Experiment 1: With vs without caching (3 actions) — target: $>2\times$ speedup
- Experiment 2: Partition count (1, 4, 8, 16) — find the sweet spot
- Experiment 3: Broadcast vs shuffle join — measure network savings
- Record timings + Web UI screenshots for each experiment

Milestone M3 Reminder

Due end of Week 8: RDD + DataFrame + MLlib analytics on Chicago Crimes.

Instructor Notes

Let me preview what is coming next and describe today's lab. In Week 8B, we move into Spark MLlib, which is machine learning at scale. You will learn the Pipeline API with Transformers and Estimators, do feature engineering with `StringIndexer` and `VectorAssembler`, and train a Random Forest to predict whether a crime leads to an arrest. That is the ML component of your M3 milestone. But first, today's lab. You have three experiments to run. Experiment one: run three actions with and without caching, and measure the time difference. You should see at least a two-times speedup. Experiment two: try different partition counts — one, four, eight, and sixteen — and find the sweet spot for our cluster. Experiment three: compare a shuffle join to a broadcast join and measure the time savings. For each experiment, record your timings and take screenshots of the Web UI. These screenshots go into your M3 report. And remember, M3 is due at the end of Week 8, so start working on it today. Use the optimized script template we just went through as your starting point.